

WIFI AP Mode

WIFI AP Mode

- 1. WiFi AP Overview
- 2. Core Concepts
 - 2.1 AP Mode Network Topology
 - 2.2 AP Configuration Parameters
 - 2.3 WPA2 Security Authentication
- 3. Project Architecture and Flow
 - Event Handling Flow
- 4. Hardware Dependencies
- 5. Source Code Analysis
 - 5.1 Header Files and Macro Configuration
 - 5.2 wifi_event_handler() — Event Callback
 - 5.3 wifi_init_ap() — AP Initialization
 - 5.4 app_main() — Main Entry
- 6. Operation Flow
 - 6.1 Compilation and Flashing Flow
 - 6.2 Normal Startup
 - 6.3 Device Connection and Disconnection
- 7. Common Problems

1. WiFi AP Overview

WiFi AP (Access Point, Hotspot Mode) allows ESP32-S3 to become its own **wireless router/hotspot**, other devices (phones, computers, another ESP32) can search for and connect to this hotspot, enabling LAN communication.

AP mode has **irreplaceable application value** in IoT. First, it is **the foundation of provisioning technology** - when device is first deployed without available WiFi network, device runs in AP mode first, user's phone connects to hotspot then sends router password through webpage or App, device then switches to STA mode to connect (first step of SmartConfig/Web provisioning flow). Second, in **scenarios without router** (field operations, site inspection, temporary exhibitions), AP mode allows users to directly connect point-to-point to device for parameter configuration, status viewing and firmware upgrade, without relying on any network infrastructure. Additionally, AP mode is widely used in **LAN multi-person collaboration** scenarios - multiple phones or computers connect to the same ESP32-S3 hotspot for local chat, file sharing, multi-player games and other LAN applications.

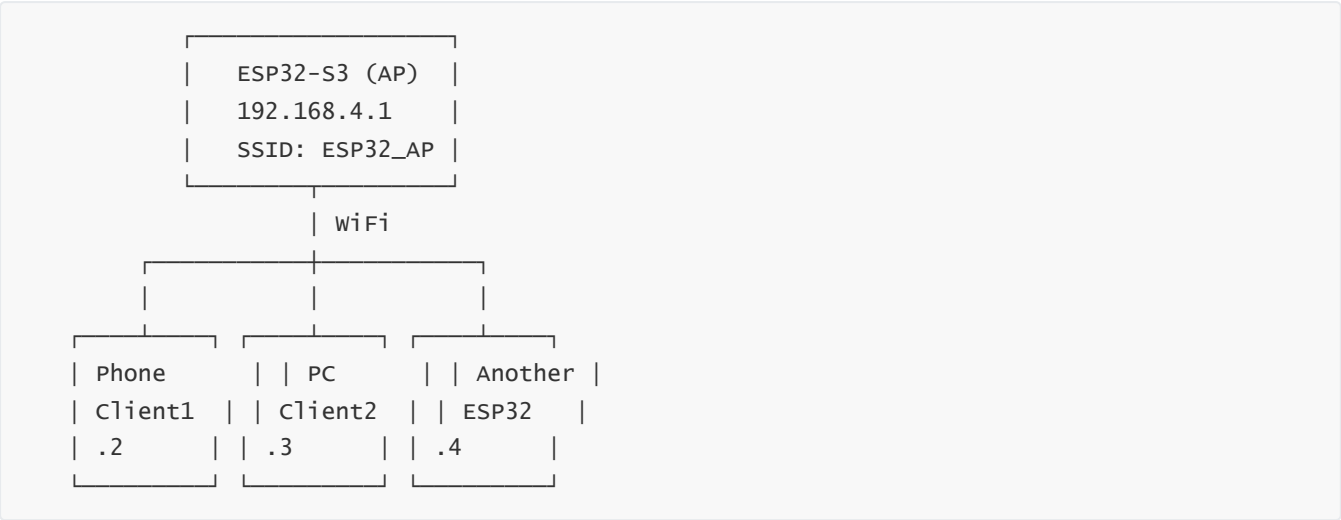
Comparison	AP Mode	STA Mode
Role	Hotspot (others connect to it)	Client (connect to others)
IP Address	Fixed 192.168.4.1	DHCP dynamic assignment
DHCP Service	ESP32 provides (assigns IP to connectors)	Router provides
Max Connections	Configurable (default 4)	N/A

Comparison	AP Mode	STA Mode
Typical Scenario	Provisioning, direct control, offline LAN	IoT internet access, data reporting

This project implements **WiFi AP mode**, creating a WPA2 encrypted hotspot named `ESP32_AP` with password `12345678`, other devices can access ESP32 via `192.168.4.1` after connecting.

2. Core Concepts

2.1 AP Mode Network Topology



2.2 AP Configuration Parameters

```
typedef struct {
    char    ssid[32];           // Hotspot name (max 32 bytes)
    char    password[64];      // Hotspot password (WPA2 requires ≥ 8 chars)
    uint8_t ssid_len;          // Actual SSID length (0=auto, default)
    uint8_t channel;           // WiFi channel (1~13, select idle channel to avoid
interference)
    uint8_t authmode;          // Encryption mode (Open/WPA/WPA2/WPA3)
    // Max simultaneous connections (default 4, max 10)
    uint8_t max_connection;
} wifi_ap_config_t;
```

2.3 WPA2 Security Authentication

Auth Mode	Description
<code>WIFI_AUTH_OPEN</code>	No password (not recommended)
<code>WIFI_AUTH_WEP</code>	Deprecated, insecure
<code>WIFI_AUTH_WPA2_PSK</code>	Recommended — Personal pre-shared key mode
<code>WIFI_AUTH_WPA3_PSK</code>	More secure, but some older devices incompatible

Auth Mode	Description
WIFI_AUTH_WPA2_Enterprise	Enterprise-grade

3. Project Architecture and Flow

```
app_main()
├─ nvs_flash_init()           ← Initialize NVS flash
├─
├─ wifi_init_ap()             ← WiFi AP initialization + start
│   ├─ esp_netif_init()       → Initialize TCP/IP network stack
│   ├─ esp_event_loop_create_default() → Create default event loop
│   ├─ esp_netif_create_default_wifi_ap() → Create AP network interface
│   ├─ esp_wifi_init()         → Initialize WiFi driver
│   ├─ Register event callbacks (WIFI_EVENT / ESP_EVENT_ANY_ID)
│   ├─ Configure AP parameters (SSID/PASS/CHANNEL/MAX_CONN/AUTH)
│   ├─ esp_wifi_set_mode(WIFI_MODE_AP) → Set AP mode
│   ├─ esp_wifi_set_config()    → Write config
│   ├─ esp_wifi_start()        → Start hotspot
│   └─ Print SSID + password + IP info
```

Event Handling Flow

```
WIFI_EVENT_AP_START
→ ESP_LOGI "WiFi AP started"           // Hotspot created

WIFI_EVENT_AP_STA_CONNECTED
→ ESP_LOGI "Station connected"         // Device connected

WIFI_EVENT_AP_STA_DISCONNECTED
→ ESP_LOGI "Station disconnected"       // Device disconnected
```

4. Hardware Dependencies

- **Hardware Connection:** No external wiring, using on-board WiFi antenna
- **Dependent Components:** `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`, `FreeRTOS`, `esp_log`

5. Source Code Analysis

Complete source code: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `2.WiFi_AP`

5.1 Header Files and Macro Configuration

Introduce string header for calculating SSID length - AP mode must explicitly tell protocol stack the byte count of hotspot name. Remaining headers are standard combination of WiFi driver, event system, NVS flash. Macros centralize four core parameters: hotspot name, password, max connections and channel at file top for easy modification.

```
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_log.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "nvs_flash.h"
#include "esp_netif.h"

#define TAG                "WIFI_AP"

// ===== AP Hotspot Config =====
#define AP_SSID            "ESP32_AP"        // Hotspot name
// Hotspot password (WPA2 ≥ 8 chars)
#define AP_PASS            "12345678"
#define AP_MAX_CONN        4                // Max connections
#define AP_CHANNEL         1                // WiFi channel
```

5.2 wifi_event_handler() — Event Callback

AP mode event handling is much simpler than STA mode - callback only needs to manage three cases: hotspot created, device connected, device disconnected. AP doesn't need to get dynamic IP from router like STA - AP's own IP is fixed at 192.168.4.1, therefore no need to register IP event callback. Each event just logs it, upper layer HTTP/TCP service depends on these events to determine device connection status.

```
static void wifi_event_handler(void* arg, esp_event_base_t event_base,
                              int32_t event_id, void* event_data)
{
    // Hotspot created successfully
    if (event_id == WIFI_EVENT_AP_START)
    {
        ESP_LOGI(TAG, "WiFi AP started");
    }
    // Device disconnected
    else if (event_id == WIFI_EVENT_AP_STADISCONNECTED)
    {
        ESP_LOGI(TAG, "Station disconnected");
    }
    // Device connected
    else if (event_id == WIFI_EVENT_AP_STACONNECTED)
    {
        ESP_LOGI(TAG, "Station connected");
    }
}
```

```
}
```

5.3 wifi_init_ap() — AP Initialization

WiFi AP initialization flow is the same as STA - network stack initialization, event loop creation, network interface creation, WiFi driver initialization, fixed order. But there are three key differences:

- Network interface uses AP version creation function, not STA version, this determines ESP32 runs as hotspot not client.
- Event callback only registers WiFi events, no IP events - AP's IP is fixed at 192.168.4.1, doesn't depend on DHCP assignment.
- Configuration struct fills AP members. SSID length field must explicitly tell protocol stack the byte count of the string, different from STA mode where system calculates automatically.

Note that there's no need to call connection function - hotspot takes effect automatically after starting, just wait for other devices to connect.

```
void wifi_init_ap(void)
{
    esp_netif_init();
    esp_event_loop_create_default();
    esp_netif_create_default_wifi_ap();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    esp_wifi_init(&cfg);

    // Register WiFi event callback
    esp_event_handler_instance_register(WIFI_EVENT,
                                        ESP_EVENT_ANY_ID,
                                        &wifi_event_handler,
                                        NULL,
                                        NULL);

    wifi_config_t wifi_config = {
        .ap = {
            .ssid = AP_SSID,                // Hotspot name
            .password = AP_PASS,            // Password
            .ssid_len = strlen(AP_SSID),
            .channel = AP_CHANNEL,          // Channel
            // Max connections
            .max_connection = AP_MAX_CONN,
            .authmode = WIFI_AUTH_WPA2_PSK
        }
    };

    // Set hotspot mode
    esp_wifi_set_mode(WIFI_MODE_AP);
    esp_wifi_set_config(WIFI_IF_AP, &wifi_config); // Apply config
    esp_wifi_start();                               // Start hotspot
    // Log SSID and password
    ESP_LOGI(TAG, "SSID: %s, PASS: %s", AP_SSID, AP_PASS);
    ESP_LOGI(TAG, "ESP32 IP: 192.168.4.1");        // AP fixed IP
}
```

```
}
```

5.4 app_main() — Main Entry

Main entry function does two things - first initialize NVS flash (required prerequisite for WiFi driver), then call initialization function to start hotspot. Final infinite loop ensures program doesn't exit, actual hotspot management and device connection are all handled asynchronously in event callback.

```
void app_main(void)
{
    nvs_flash_init();           // Initialize NVS flash
    wifi_init_ap();            // Start AP mode

    while (1)
    {
        // Keep program running
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

6. Operation Flow

6.1 Compilation and Flashing Flow

ESP-IDF flashing flow: See `ESP-IDF → 1. Before Development → 3. Build and Flash`.

6.2 Normal Startup

After flashing, reset development board, serial assistant will output the following - `SSID/PASS` confirms hotspot started, `DHCP server started` means DHCP server ready to assign IPs to clients, `WiFi AP started` is user log from event callback, last line emphasizes AP fixed IP:

Log field meanings:

Field	Example	Meaning
<code>I</code>	<code>I</code>	INFO level log
<code>WIFI_AP</code>	<code>WIFI_AP</code>	Log tag
<code>esp_netif_lwip</code>	<code>esp_netif_lwip</code>	Network interface DHCP service log
<code>SSID:</code>	<code>ESP32_AP</code>	Hotspot name
<code>PASS:</code>	<code>12345678</code>	Hotspot password
<code>DHCP server started</code>	<code>DHCP server started</code>	DHCP server started, begins assigning IP to connected devices
<code>WiFi AP started</code>	<code>WiFi AP started</code>	WiFi AP hotspot successfully created

Field	Example	Meaning
ESP32 IP:	192.168.4.1	AP fixed IP address

```
I (551) WIFI_AP: SSID: ESP32_AP, PASS: 12345678
I (551) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (571) WIFI_AP: WiFi AP started
I (571) WIFI_AP: ESP32 IP: 192.168.4.1
```

```
I (551) WIFI_AP: SSID: ESP32_AP, PASS: 12345678
I (551) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (571) WIFI_AP: WiFi AP started
I (571) WIFI_AP: ESP32 IP: 192.168.4.1
```

6.3 Device Connection and Disconnection

Search WiFi with phone, find `ESP32_AP`, enter password `12345678` to connect. Connection process outputs in order - `Station connected` is association success log from user code callback, then `wifi:new:<1,1>` indicates driver internal state changed from "no associated devices" to "1 associated device", two `wifi:<ba-add>` are link layer Block Ack negotiation complete, finally `DHCP server assigned IP` means DHCP has assigned IP to client, at which point device can communicate normally:

Log field meanings:

Field	Example	Meaning
Station connected	Station connected	Device associated to hotspot (user code event callback output)
wifi:new:<1,1>	new:<1,1>	WiFi driver state transition: associated devices count from 0 to 1 (IDF underlying log)
wifi:<ba-add>	ba-add	WiFi driver Block Ack negotiation successful, link layer ready (IDF underlying log)
DHCP server assigned IP	192.168.4.2	DHCP server assigned IP address to client, device can communicate normally

```
I (11061) WIFI_AP: Station connected
I (11091) wifi:new:<1,1>, old:<1,0>, ap:<1,1>, sta:<255,255>, prof:1
I (11111) wifi:<ba-add>idx:2 (ifx:1, e6:d1:24:85:df:49), tid:0, ssn:0, winSize:64
I (11121) wifi:<ba-add>idx:3 (ifx:1, e6:d1:24:85:df:49), tid:7, ssn:0, winSize:64
I (11151) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
```

```
I (11061) WIFI_AP: Station connected
I (11111) wifi:<ba-add>idx:2 (ifx:1, e6:d1:24:85:df:49), tid:0, ssn:0, winSize:64
I (11121) wifi:<ba-add>idx:3 (ifx:1, e6:d1:24:85:df:49), tid:7, ssn:0, winSize:64
I (11151) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
I (27471) wifi:<ba-add>idx:4 (ifx:1, e6:d1:24:85:df:49), tid:5, ssn:0, winSize:64
I (256861) wifi:station: e6:d1:24:85:df:49 leave, AID = 1, reason = 3, bss_flags is 756835, bss:0x3fcad4f8
I (256871) wifi:new:<1,0>, old:<1,1>, ap:<1,1>, sta:<255,255>, prof:1, snd_ch_cfg:0x0
```

After disconnection, log shows complete link teardown process - three `wifi:<ba-del>` are driver deleting Block Ack sessions one by one, releasing link layer resources, `wifi:new:<1,0>` indicates driver internal state from "1 associated device" to "0 devices", finally `station disconnected` is user code event callback confirming disconnect:

Field	Example	Meaning
<code>wifi:<ba-del></code>	<code>ba-del</code>	WiFi driver deleted Block Ack session, link layer released resources (IDF underlying log)
<code>wifi:new:<1,0></code>	<code>new:<1,0></code>	WiFi driver state transition: associated devices from 1 to 0 (IDF underlying log)
<code>Station disconnected</code>	<code>Station disconnected</code>	Client disconnected from hotspot (user code event callback output)

```
I (256871) wifi:<ba-del>idx:2, tid:0
I (256871) wifi:<ba-del>idx:4, tid:5
I (256871) wifi:new:<1,0>, old:<1,1>, ap:<1,1>, sta:<255,255>, prof:1, snd_ch_cfg:0x0
I (256881) wifi:<ba-del>idx:3, tid:7
I (256891) WIFI_AP: Station disconnected
```

```
(256871) wifi:<ba-del>idx:2, tid:0
(256871) wifi:<ba-del>idx:4, tid:5
(256881) wifi:<ba-del>idx:3, tid:7
(256881) wifi:new:<1,1>, old:<1,0>, ap:<1,1>, sta:<255,255>, prof:1, snd_ch_cfg:0x0
(256891) WIFI_AP: Station disconnected
```

7. Common Problems

Symptom	Cause	Solution
Phone can't find hotspot	WiFi not started normally / channel occupied	Check serial logs confirm <code>WiFi AP started</code> ; try changing <code>AP_CHANNEL</code> (e.g., 6, 11)
"Password incorrect" when connecting	Password less than 8 chars or SSID abnormal	WPA2 requires password ≥ 8 chars; confirm <code>AP_PASS</code> length is sufficient
Can't access <code>192.168.4.1</code> after connecting	No HTTP/TCP server running	AP is just network layer, need upper layer service listening on port to respond
New device can't connect after multiple devices connected	<code>AP_MAX_CONN</code> too small	Increase <code>AP_MAX_CONN</code> (max 10), but note memory consumption
Hotspot unstable, frequent disconnections	2.4G channel interference	Use WiFi channel scan tool to select idle channel, modify <code>AP_CHANNEL</code>

Symptom	Cause	Solution
<code>WIFI_EVENT_AP_START</code> no log	NVS not initialized	Ensure <code>nvs_flash_init()</code> called at beginning of <code>app_main()</code>